claude-windows / c1fcb4aa-537d-41fd-858e-53f93349bf5a

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\subagents\workflows\wf_7049e1ed-b93\agent-ab28ef5be5e8cc304.jsonl`
- SHA-256: `afedb9a90b4e44645f0800a7173b8a5ea48c2837d747901f689e155a4b678460`
- Source modified: `2026-07-02T04:53:47+00:00`
- Imported at: `2026-07-05T16:48:24+00:00`
- project: `wf_7049e1ed-b93`
- session_id: `c1fcb4aa-537d-41fd-858e-53f93349bf5a`

Transcript

1. user (2026-07-02T04:45:39.420Z)

```
You are a senior engineer producing ONE isolated PR. Today is 2026-07-02.
REPO: C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer (a shared checkout ? follow the safety rules exactly).
AUDIT FINDINGS YOU ARE FIXING: R1-16 (gdrive_api RAM buffering + unsafe put ordering) ? Grep C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-indexer/c1fcb4aa-537d-41fd-858e-53f93349bf5a/scratchpad/AUDIT_REPORT_khelsutra-guru_2026-07-02.md for these IDs and read the full evidence + verifier notes before coding.
TASK BRIEF:
In C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer:
backend/storage/gdrive_api.py ? (1) fetch_to_local at ~:144 buffers the whole file via get_media().execute(): switch to chunked googleapiclient.http.MediaIoBaseDownload to a file handle (lazy import stays inside the method per the file's convention); (2) put() currently deletes the existing object BEFORE the new upload succeeds: reorder to upload-new-first (or temp-name then swap) and delete the old only after success, preserving the idempotent-put contract from PR #338. Extend the injected-fake tests: large-file chunked download path, put failure mid-upload leaves the old object intact.

BRANCHING SAFETY (mandatory ? shared checkout, sibling sessions exist):
1. In the MAIN repo checkout run: git fetch origin
2. Create an ISOLATED WORKTREE: git worktree add "C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-indexer/c1fcb4aa-537d-41fd-858e-53f93349bf5a/scratchpad/wt-<shortname>" -b <branch-name> origin/master   (branch explicitly FROM origin/master ? never from the current local branch; use origin/main for repos whose default is main)
3. Do ALL work inside the worktree. Before committing: git branch --show-current + git log --oneline -1 must show YOUR branch on the origin-master base.
4. Stage EXPLICIT paths only (git add <file> <file>) ? NEVER git add -A / . / -u
5. Before opening the PR: git log --oneline origin/master..HEAD must list ONLY your commits.
COMMIT/PR CONVENTIONS:
- Commit message: conventional style matching the repo's history; end the message with a blank line then: Co-Authored-By: Claude Fable 5 <noreply@anthropic.com>
- PR body: what/why, a "## Verification" section stating exactly what you ran locally and what CI covers, and end with: ? Generated with [Claude Code](https://claude.com/claude-code)
- Do NOT merge the PR. Do NOT remove the worktree (a fixer may need it). Return the PR number, branch, worktree path.
QUALITY BAR:
- Read the repo's CLAUDE.md first; for the engine also docs/CODE_MAP.md §0 (file placement) before adding any new file.
- Add/extend tests for every behavior change. Run the targeted tests you touched (python -m pytest tests/<relevant> -q) + ruff check on changed files; if mypy config exists, run mypy on touched modules. Full suite is CI's job ? do not run it.
- Default-safe: new knobs default OFF / behavior-preserving unless the task explicitly says otherwise.
- The full audit evidence is in C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-
```

Projects-badminton-highlight-indexer/c1fcb4aa-537d-41fd-858e-53f93349bf5a/scratchpad/AUDIT_REPORT_khelsutra-guru_2026-07-02.md ? Grep it for your finding IDs and READ the verifier notes; they contain corrections (exact file:line, scoping traps) you MUST honor.

If the brief's premise turns out false against current code (something already fixed, file moved), adapt honestly and say so in notes ? never force a pointless diff. If you genuinely cannot complete (env broken, auth missing), open NO PR and explain in notes with pr_number=0.

2. user (2026-07-02T04:45:46.762Z)

157-[Omitted long context line]
158-[Omitted long context line]
159-
160-### R1-13 [P2/S] doc-rot ? badminton-highlight-indexer
161-**BACKLOG.md lists three shipped items as open, including a red 'blocks something' ByteTrack entry**
162-- anchor: `docs/BACKLOG.md:39` · tracked in: ? · finder: engine:audit-doc-reconcile
163-[Omitted long context line]
164-- why it matters: This stale state has already caused real confusion: the project memory feeding this very audit listed D13 (ByteTrack) as a remaining item. A red-flagged 'blocks something' entry that is actually done distorts session-start prioritization for every future session.
165-- proposed fix: Docs-only PR: flip the three entries to done with PR links (#386, #370, #380), matching the checked-off convention already used at BACKLOG.md:24. Can ride in the same reconcile PR as the DOC_STATUS/README fixes.
166-[Omitted long context line]
167-[Omitted long context line]
168-
169-### R1-14 [P2/S] doc-rot ? badminton-highlight-indexer
170-**DOC_STATUS register + docs/README index are ~15 PRs behind the cleanup-audit and decision-snapshot trackers**
171-- anchor: `docs/DOC_STATUS.md:47` · tracked in: docs/CODE_CLEANUP_AUDIT_2026-06-24.md §8 DoD (unchecked 'docs/README.md updated' item) · finder: engine:audit-doc-reconcile
172-[Omitted long context line]
173-- why it matters: DOC_STATUS is by convention the archival/lifecycle board and README is the ramp-up index; both are read at every session start. When they contradict the per-doc source-of-truth trackers, sessions either relitigate finished work or trust stale gating claims. The cleanup audit's own DoD (line 225) is blocked on exactly this README update.
174-- proposed fix: One docs-only reconcile PR: refresh DOC_STATUS.md:47 row (Phase 2 nearly done, remaining = P10 + P16+), add a DECISION_SNAPSHOT_REGRESSION row, fix docs/README.md:41-42 blurbs, and rewrite the NEXT_STEPS.md:6 'Where we are' paragraph to the post-06-21 state.
175-[Omitted long context line]
176-[Omitted long context line]
177-
178-### R1-15 [P2/M] test-gap ? badminton-highlight-indexer
179-**M4 quality-floor gate still missing ? the one unblocked item keeping the golden-fixtures track from DONE**
180-- anchor: `docs/GOLDEN_REGRESSION_FIXTURES.md:156` · tracked in: docs/GOLDEN_REGRESSION_FIXTURES.md §7 M4; docs/BACKLOG.md:60 · finder: engine:audit-doc-reconcile
181-- evidence: GOLDEN_REGRESSION_FIXTURES.md:156 (M4 row) is ?; verified tests/test_golden_quality_floor.py and eval_baselines/regression_baseline.json do not exist (ls: No such file). §9 DoD (line 197) requires M4 before the track can flip DONE. BACKLOG.md:60 calls it 'recommended-next; unblocked... the cheapest step to flip the track to DONE' (advice standing since ~06-16).
182-- why it matters: Today the golden suite is characterization-only: it catches behavior CHANGES but has no committed F1 floor or windowing fingerprint, so a deliberate re-freeze after a SERVED retune could silently lock in worse quality. It is also the sole unblocked blocker to archiving the whole fixtures project.
183-- proposed fix: Implement M4 per the doc's own recipe: seed per-slug floors from the F1 already frozen in each fx_*/expected.json score (floor = current - tolerance), record a windowing fingerprint in eval_baselines/regression_baseline.json, keep CONTROL-only, add to the golden-crossos CI matrix.
184-[Omitted long context line]

185-[Omitted long context line]
186-
187:### R1-16 [P2/L] tech-debt ? badminton-highlight-indexer
188-**Decision-snapshot regression track: decisions locked 06-25, zero code shipped since; tracker P0 row also stale**
189-- anchor: `docs/DECISION_SNAPSHOT_REGRESSION.md:66` · tracked in: docs/DECISION_SNAPSHOT_REGRESSION.md §7 (P1-P8) · finder: engine:audit-doc-reconcile
190-- evidence: §7 tracker (lines 63-73): P1-P8 all ?. Verified in code: grep 'build_stitch_plan' across backend/ and tests/ returns nothing (P2's pure stitch-plan seam was never factored out of backend/pipeline/stitcher/compiler.py), and tests/test_golden_fixtures.py has no stitch-plan snapshot. Minor rot: the P0 row (line 65) still shows ? '(this PR)' though the doc merged as #383 (commit 1283e18, 2026-06-25) and the header still says 'DRAFT'.
191-- why it matters: This gate exists precisely to bless genuine (non-AST-identical) refactors as quality-neutral ? i.e., exactly the class of change as the stalled #390 main.py split. Executing P1-P4 first would de-risk redoing the god-module split; leaving it a week-stale locked-decisions doc means refactors continue with no behavioral proof.
192-- proposed fix: Start §7 P1+P2 (canonical decision-snapshot schema + factor pure build_stitch_plan() out of compiler.py) as the next Claude-doable engine work; flip the P0 row to ? #383 and status to IN PROGRESS in the same PR. Ideally land P1-P4 before re-cutting the main.py split.
193-[Omitted long context line]
194-[Omitted long context line]
195-
196-### R1-17 [P2/M] tech-debt ? badminton-highlight-indexer
197-**Fetched-input scratch copies are never reclaimed anywhere (process jobs, compile jobs, /api/assets, cloud_run marker reads) ? 'OS temp reclamation' premise is false on Windows**
198-- anchor: `backend/storage/__init__.py:89` · tracked in: ? · finder: engine:correctness
199-[Omitted long context line]
200-- why it matters: Slow-burn disk exhaustion on exactly the appliance the product ships as; also inflates the free-space guard's false rejections.
201-- proposed fix: Wrap remote-input acquisition in a per-job scope that deletes the scratch dir in a finally (worker already owns the job lifecycle), or route scratch under output_base(cfg)/tmp with an age-based sweep at startup; make cloud_run._read_json use TemporaryDirectory.
202-[Omitted long context line]
203-[Omitted long context line]
204-
205-### R1-18 [P2/S] critical-fix ? badminton-highlight-indexer
206-**GDriveApiStorage.fetch_to_local buffers the entire file in RAM (get_media().execute()); put() deletes the old object before the new upload succeeds**
207-- anchor: `backend/storage/gdrive_api.py:144` · tracked in: ? · finder: engine:correctness
208-- evidence: gdrive_api.py:143-148: `data = self._service.files().get_media(fileId=file_id).execute()` then `fh.write(data)` ? the whole object is materialized as one bytes value (no MediaIoBaseDownload streaming). gdrive_api.py:163-170: put() resolves an existing file at the path and `files().delete(...)` BEFORE `files().create(...)` ? delete-then-create is not atomic.
209-- why it matters: Latent OOM + data-loss on the Drive-API storage hot path before it goes live for the reel round-trip.
210-- proposed fix: Stream downloads with googleapiclient.http.MediaIoBaseDownload in chunks; for put(), create the new file first (or upload to a temp name) and delete the old one only after success.
211-[Omitted long context line]
212-[Omitted long context line]
213-
214-### R1-19 [P2/S] critical-fix ? badminton-highlight-indexer
215-**config.json output.output_dir is silently clobbered by the CLI --output-dir DEFAULT on every server/CLI start ? the typed knob is dead, and the DB path moves with it**
216-- anchor: `backend/main.py:1992` · tracked in: ? · finder: engine:correctness
217-- evidence: main.py:1837-1841 defines `--output-dir` with default='output'; main.py:1992 unconditionally does `global_config.output.output_dir = resolve_output_dir(args.output_dir)` ? the override fires even when the flag was never passed, so a config.json value can never take effect. models.py:274-276 documents the knob as if it's honored ('The CLI's --output-dir overrides this at startup' ? implying only when given). db_path derives from it (main.py:1995-1997).

3. user (2026-07-02T04:45:47.396Z)

```
1       # CLAUDE.md ? project memory / agent handoff
2
3       Local, AI-powered **badminton (? racket-sports) highlight indexer + rally detector**:
4       FastAPI + SQLite + ffmpeg + GPU CV/AI, with paid **Gemini** as the cloud AI. Sibling
5       repos: `sports-data-collector`, a VLC-based `rally-annotator`.
6
7       ## Read these first
8       - **`docs/README.md`** ? doc index. **`docs/CODE_MAP.md`** ? code navigation + data
9         contracts (the cold-start map; read before touching code).
10      - **Authoritative owned-model roadmap:** `docs/OWNED_MODEL_IMPLEMENTATION_PLAN.md` (+
the
11        live `docs/NEXT_STEPS.md`). These SUPERSEDE the 2026-06-07 strategy docs below where
they
12        disagree (licensing, Gemma-4 stance, base model, conversational-QA).
13      - **Strategy (post-scaffolding):** `docs/COMMERCIALIZATION.md`,
`docs/PMF_MARKET_RESEARCH.md`,
14        `docs/PLATFORM_ARCHITECTURE.md`, `docs/DEFERRED_HARDENING_PLAN.md`. Also
`docs/ROADMAP.md`
15        (offline-segmenter narrative) and `docs/BACKLOG.md`.
16      - **Rally-detection quality track (design merged 2026-06-13, owner-gated, implementation
in progress):**
17        `docs/HOW_RALLY_DETECTION_WORKS.md` (2-layer mental model) ?
`docs/MULTI_SIGNAL_FUSION_PLAN.md`
18        (fuse shuttle + person + audio cues; the held-shuttle bug is the already-built
`rally_gate.py`
19        "Gate A" **not wired into production** ? cheapest win) ? `docs/EXPERIMENT_HARNESS.md`
(A/B + shadow
20        eval so cues land flag-gated/default-OFF with zero regression) ?
`docs/ROORA_LABELING_GUIDELINES.md`
21        (v8 vendor labeling spec). **`docs/NEXT_STEPS.md` has the prioritized pick-up for this
track.**
22      - **History:** `docs/archives/` ? ADRs (`decisions/DECISIONS.md`), research, and
23        **`checkpoints/`** (latest: `2026-06-strategy-foundation/`). **Archive policy:** only
move
24        **terminal-state** (DONE/REJECTED) work; live docs stay at `docs/` top level ? and
**before
25        archiving, HONESTLY update the doc first** (verify claims vs code ? flip status + a
completion
26        note ? update inbound refs ? keep the lineage ? then `git mv`), per the checklist +
living
27        lifecycle register in **`docs/DOC_STATUS.md`**.
28
29      ## Current state (June 2026)
30      - **Scaffolding complete**; strategy foundation + owned-model roadmap merged. **No
31        platform/owned-model implementation has started.** A 2026-06-10 audit-driven hardening
32        sweep landed (licensing gate, detector keying, re-ingest safety, config, eval gate,
API
33        validation, reliability, CI + test coverage) ? see git history.
34      - **Two tracks, both owner-gated:** the **committed next PLATFORM slice = async job
model**
35        (`DEFERRED_HARDENING_PLAN.md` #16, single-tenant); the **owned-model track** starts at
a
36        greenlit Phase-0 milestone (`OWNED_MODEL_IMPLEMENTATION_PLAN.md`). **Do not start
37        implementation without explicit owner approval.**
38
39      ## Architecture in one breath
40      - **Pluggable registries** (ABC + `Registry` singleton): segmenters / providers /
validators
41        / sports / annotations ? see `backend/pipeline/segmenters/base.py`. *Future:*
42        `StorageBackend` + `ComputeBackend` registries (PLATFORM §3).
43      - `video_id` = MD5 of the file ? `backend/utils/hashing.py::compute_video_id` (one
helper).
44      - Typed config = `backend/config/` (Pydantic). DB schema evolves via the `PRAGMA
```

```
45        user_version` migration ladder in `backend/infrastructure/database.py`.
46
47      ## Committed guardrails (non-negotiable)
48      - **Paid LLMs (Gemini/OpenAI/Anthropic) = inference / serving / fallback ONLY ? never a
49        training data source** (terms/copyright). Commercial-clean licensing for EVERY
dependency ?
50        code, data, AND weights: **MIT / Apache-2.0 / BSD only** (ratified 2026-06-09, owner-
agreed).
51      - **Owned-model base:** **Qwen3-VL (Apache-2.0)** primary, InternVL3 (MIT) fallback;
**Gemma-4
52        is AMBER / bench-only** (Apache grant likely but Prohibited-Use posture unconfirmed ?
counsel
53        needed; do NOT build on it without sign-off). **Reject Gemma 3** (viral license).
**Exclude
54        minors** from the training pool; per-user training data needs a separate explicit opt-
in.
55      - **v1 = rally + highlight + history + correction loop**; **gait/biometrics strictly
future**
56        (GDPR Art. 9).
57      - **Generic data schema**; `coach ? athlete` is an **optional overlay**, not baked in.
58      - **Compute = GCP ONLY (ADR-013, 2026-06-20):** GPU training + serving run on **GCP**
(`deploy/gcp/`;
59        Cloud Run+GPU for serving per ADR-012). **DigitalOcean is DROPPED for compute** (DO
GPU not enabled
60        on the account + not cleanly credit-funded; Paperspace is subscription-gated). Don't
re-evaluate
61        DO/Paperspace for compute without a new ADR; the `$200` DO credits go to **non-compute
only**. The
62        `deploy/digitalocean/*` scripts are kept solely as provider-agnostic Linux-GPU setup
reused by GCP.
63
64      ## Workflow conventions
65      - **Branch off `master`; ONE PR per work item; NO PR stacking.** Don't open a PR unless
asked.
66      - Tests: `python run_all_tests.py` (offline, fully mocked). Python 3.10+ (the `trackers`
package requires ?3.10); `ffmpeg` required.
67      - Never push to the default branch directly; descriptive commit messages.
68      - **Substantial design/project work gets a tracked project doc** following
`docs/PROJECT_DOC_TEMPLATE.md` (status header · decisions-locked · progress tracker ·
definition-of-done; archived on completion). Precedents: `docs/RALLY_QUALITY_RESEARCH.md` §7,
`docs/GOLDEN_REGRESSION_FIXTURES.md`.
69      - **Launch / "bit-flip" PRs** (a default-flip that drives a tracked project to terminal
state) MUST, in the SAME PR, run the `docs/DOC_STATUS.md` §2 archival due-diligence on that
project's docs (verify-vs-code ? flip status + completion note ? update inbound refs ? keep ADR
lineage ? `git mv` to `docs/archives/` ? record in DOC_STATUS §3), and fix any internal-link
breakage the move causes. Enforced by `.github/PULL_REQUEST_TEMPLATE.md` + the `link-check` CI
job (`python -m tools.check_md_links`).
70      - ?? Hand the heaviest machine-side checks (real video, the full GPU/WSL/WASB suite) to
the owner,
71        who has the golden corpus + labelled videos; verify pure-logic paths locally.
72
```

4. user (2026-07-02T04:45:59.396Z)

```
master
Already up to date.
e410136 api: /api/videos surfaces fps + resolution (from the stored ffprobe details) (#413)
```

5. user (2026-07-02T04:45:59.413Z)

```
1       """Google Drive **API** backend (OAuth / service-account) ? ``gdrive-api://`` (M12, D14
Northstar).
2
3       The cloud-native Drive round-trip: read a source from + write the stitched reel **back
to** a user's
```

```
4       Google Drive **via the Drive API** ? no local mount required (unlike the ``gdrive://``
backend, which
5       is plain local-FS over a *mounted* Drive for Desktop and stays untouched). This is the
headless path a
6       Cloud Run / GPU box uses.
7
8       ``gdrive-api://<path>`` names a path **relative to a root folder** (default the Drive
``root``, or
9       ``storage.gdrive-api.root_folder_id``); the backend resolves each path component to a
Drive file id via
10      ``files().list``. The Google libraries (``google-api-python-client`` + ``google-auth``)
are imported
11      **lazily** ? ``import backend.storage`` never pulls them ? and a ``service`` may be
**injected** for
12      fully-offline tests (mirroring ``GCSStorage(client=...)``).
13
14      ? **Credentials are never in config.** The real service is built from Application
Default Credentials
15      (a service account / workload identity) over the ``drive`` scope. **Per-user interactive
OAuth (the
16      consumer "connect your Drive" flow: client id/secret + refresh tokens) is OWNER-gated**
? this backend
17      ships the service-account/ADC build + the injected fake; the live per-user OAuth app is
a follow-up.
18
19      **Scope caveats (M12):** handles **binary** files (videos / reels / JSON) ? Google-
native Docs
20      (Sheets/Slides) have no direct download and would need ``files().export_media`` (out of
scope; export
21      them first). ``put`` is idempotent (deletes an existing file at the same path before
creating). Path
22      resolution keys on ``(name, parent)``; if Drive already holds duplicate-named files
under one folder
23      (a rare pre-existing state this backend never creates), resolution returns the first
match.
24      """
25
26      from __future__ import annotations
27
28      import io
29      import os
30      from datetime import datetime
31      from typing import Any, Iterator, Optional
32
33      from backend.storage.base import StorageBackend
34      from backend.storage.ref import StorageRef, StorageStat
35
36      _FOLDER_MIME = "application/vnd.google-apps.folder"
37      _DRIVE_SCOPE = "https://www.googleapis.com/auth/drive"
38
39
40      def _q_escape(name: str) -> str:
41          """Escape a value for a Drive ``q`` string literal (single-quoted): ``\\`` then
``'``."""
42          return name.replace("\\", "\\\\").replace("'", "\\'")
43
44
45      class GDriveApiStorage(StorageBackend):
46          """Drive-API-backed storage. Construct with an injected ``service`` (tests) or let
it build one
47          from ADC (production). ``root_folder_id`` anchors relative ``gdrive-api://``
paths."""
48
49          def __init__(
50              self, *, service: Any = None, root_folder_id: str = "root", **_ignored
```

```
        ) -> None:
            self._root_id = root_folder_id or "root"
            if service is not None:
                self._service = service
                return
            self._service = self._build_service()

        # -- construction (real path; lazy Google imports)
-------------------------------------
        def _build_service(self) -> Any:
            try:
                import google.auth  # type: ignore
                from googleapiclient.discovery import build  # type: ignore
            except (
                ImportError
            ) as e:  # pragma: no cover - exercised only without the extra installed
                raise RuntimeError(
                    "the gdrive-api backend needs the 'storage-gdrive-api' extra "
                    "(google-api-python-client + google-auth). Install it on a host that
uses "
                    "gdrive-api:// URIs; credentials come from ADC (a service account /
workload "
                    "identity over the Drive scope) ? never from config."
                ) from e
            creds, _ = google.auth.default(scopes=[_DRIVE_SCOPE])
            return build("drive", "v3", credentials=creds, cache_discovery=False)

        def _media_body(self, local_path: str, content_type: Optional[str]) -> Any:
            """The upload media object for ``files().create`` ? lazy ``MediaFileUpload``
(real path).
            A test seam: monkeypatch this to avoid the ``googleapiclient`` dependency."""
            from googleapiclient.http import (
                MediaFileUpload,  # type: ignore  # pragma: no cover
            )

            return MediaFileUpload(
                local_path, mimetype=content_type, resumable=True
            )  # pragma: no cover

        # -- path -> id resolution
-----------------------------------------------------------------
        def _resolve_id(self, key: str, *, create_dirs: bool = False) -> Optional[str]:
            """Walk ``key``'s path components from the root folder, returning the leaf's
file id (or
            ``None`` if a component is missing). With ``create_dirs`` EVERY missing
component is created as
            a folder ? ``put`` uses this on the parent-directory path (where all components
are folders),
            so a write into a fresh subfolder works. The read paths leave it ``False`` (a
missing component
            ? ``None``), never creating anything."""
            parent = self._root_id
            parts = [p for p in key.split("/") if p]
            for part in parts:
                q = (
                    f"name = '{_q_escape(part)}' and '{_q_escape(parent)}' in parents "
                    f"and trashed = false"
                )
                found = (
                    self._service.files()
                    .list(q=q, spaces="drive", fields="files(id, mimeType)")
                    .execute()
                    .get("files", [])
                )
                if found:
```

```
                    parent = found[0]["id"]
                elif create_dirs:
                    parent = (
                        self._service.files()
                        .create(
                            body={
                                "name": part,
                                "mimeType": _FOLDER_MIME,
                                "parents": [parent],
                            },
                            fields="id",
                        )
                        .execute()["id"]
                    )
                else:
                    return None
            return parent

        def _split(self, key: str) -> tuple[str, str]:
            """Split a ``gdrive-api://`` key into (parent dir path, leaf name)."""
            clean = key.strip("/")
            head, _, name = clean.rpartition("/")
            return head, name

        # -- StorageBackend interface
-----------------------------------------------------------
        def fetch_to_local(self, ref, dst=None, *, overwrite=False) -> str:
            if dst is None:
                raise ValueError(
                    "GDriveApiStorage.fetch_to_local requires a dst path (no local mount)"
                )
            dst = str(dst)
            if os.path.exists(dst) and not overwrite:
                return dst
            file_id = self._resolve_id(ref.key)
            if not file_id:
                raise FileNotFoundError(f"gdrive-api: {ref.uri!r} not found")
            data = (
                self._service.files().get_media(fileId=file_id).execute()
            )  # alt=media ? raw bytes
            os.makedirs(os.path.dirname(dst) or ".", exist_ok=True)
            with io.open(dst, "wb") as fh:
                fh.write(data)
            return dst

        def put(self, local_path, ref, *, content_type=None) -> StorageRef:
            parent_path, name = self._split(ref.key)
            parent_id = (
                self._resolve_id(parent_path, create_dirs=True)
                if parent_path
                else self._root_id
            )
            if parent_id is None:
                raise FileNotFoundError(f"gdrive-api: cannot resolve parent of {ref.uri!r}")
            # Idempotent: Drive allows duplicate (name, parent), so a plain create() on a
re-put would
            # ORPHAN the prior file (and reads would keep returning the stale first match).
Delete an
            # existing file at this exact path first, so put() overwrites rather than
accumulates.
            existing = self._resolve_id(ref.key)
            if existing:
                self._service.files().delete(fileId=existing).execute()
            self._service.files().create(
                body={"name": name, "parents": [parent_id]},
```

```
                media_body=self._media_body(str(local_path), content_type),
                fields="id",
            ).execute()
            return ref

        def exists(self, ref) -> bool:
            return self._resolve_id(ref.key) is not None

        def stat(self, ref) -> StorageStat:
            file_id = self._resolve_id(ref.key)
            if not file_id:
                raise FileNotFoundError(f"gdrive-api: {ref.uri!r} not found")
            meta = (
                self._service.files()
                .get(fileId=file_id, fields="size, modifiedTime, md5Checksum, mimeType")
                .execute()
            )
            modified = None
            if meta.get("modifiedTime"):
                # Drive returns RFC-3339 (e.g. "2026-06-22T10:00:00.000Z"); normalize the
trailing Z.
                modified = datetime.fromisoformat(
                    meta["modifiedTime"].replace("Z", "+00:00")
                )
            return StorageStat(
                size=int(meta.get("size") or 0),
                modified=modified,
                etag=meta.get("md5Checksum"),
                content_type=meta.get("mimeType"),
            )

        def list(self, prefix) -> Iterator[StorageRef]:
            folder_id = (
                self._resolve_id(prefix.key) if prefix.key.strip("/") else self._root_id
            )
            if not folder_id:
                return
            base = prefix.key.strip("/")
            page_token: Optional[str] = None
            while True:
                resp = (
                    self._service.files()
                    .list(
                        q=f"'{_q_escape(folder_id)}' in parents and trashed = false",
                        spaces="drive",
                        fields="nextPageToken, files(id, name)",
                        pageToken=page_token,
                    )
                    .execute()
                )
                for f in resp.get("files", []):
                    key = f"{base}/{f['name']}" if base else f["name"]
                    yield StorageRef("gdrive-api", "", key, f"gdrive-api://{key}")
                page_token = resp.get("nextPageToken")
                if not page_token:
                    break

        def delete(self, ref) -> None:
            file_id = self._resolve_id(ref.key)
            if file_id:
                self._service.files().delete(fileId=file_id).execute()

```

6. user (2026-07-02T04:46:09.452Z)

```
205     ### R1-18 [P2/S] critical-fix ? badminton-highlight-indexer
206     **GDriveApiStorage.fetch_to_local buffers the entire file in RAM
(get_media().execute()); put() deletes the old object before the new upload succeeds**
207     - anchor: `backend/storage/gdrive_api.py:144` · tracked in: ? · finder:
engine:correctness
208     - evidence: gdrive_api.py:143-148: `data =
self._service.files().get_media(fileId=file_id).execute()` then `fh.write(data)` ? the whole
object is materialized as one bytes value (no MediaIoBaseDownload streaming).
gdrive_api.py:163-170: put() resolves an existing file at the path and `files().delete(...)`
BEFORE `files().create(...)` ? delete-then-create is not atomic.
209     - why it matters: Latent OOM + data-loss on the Drive-API storage hot path before it
goes live for the reel round-trip.
210     - proposed fix: Stream downloads with googleapiclient.http.MediaIoBaseDownload in
chunks; for put(), create the new file first (or upload to a temp name) and delete the old one
only after success.
211     - verifier notes: File/line accurate (line 144 is the get_media().execute() call).
Failure scenarios: (1) fetch_to_local on a multi-GB source/reel buffers the full file in RAM on
a memory-limited Cloud Run box ? OOM kill; (2) re-put() of a reel to an existing path deletes
the old file, then the resumable create fails (network/quota) ? the reel is gone with no copy at
either end. Fix refinement: for put(), prefer `files().update(fileId=existing, media_body=...)`
when a file already exists ? atomic in-place content replacement that preserves the file id and
avoids both the delete-first data-loss window and the duplicate-(name,parent) window that
create-first-then-delete would open (the file's own comment at lines 160-162 notes _resolve_id
returns the first match among duplicates). Download fix as proposed: MediaIoBaseDownload chunked
streaming.
212     - verifier reasoning: backend/storage/gdrive_api.py:143-148 ? fetch_to_local does `data
= self._service.files().get_media(fileId=file_id).execute()` then `fh.write(data)`: the whole
object is returned as a single bytes value; no MediaIoBaseDownload/chunked loop exists in the
file (only MediaFileUpload for uploads, lines 78-84). Docstring (lines 3-6, 19) states this
backend serves videos/reels on headless Cloud Run/GPU hosts, so multi-GB buffering is a genuine
OOM risk. backend/storage/gdrive_api.py:163-170 ? put() resolves `existing`, calls
`files().delete(fileId=existing).execute()`, and only then `files().create(...)`: a create
failure after the delete loses the prior object (non-atomic overwrite). git log --
backend/storage/gdrive_api.py shows only formatting/isort commits (b6214e5, f523d3a) since the
introducing commit 819e395 (M12, PR #338) ? not fixed. Not owner-gated: the module's owner gate
(lines 14-17) covers the per-user OAuth flow, not this code's download/upload mechanics.
213
214     ### R1-19 [P2/S] critical-fix ? badminton-highlight-indexer
```

7. user (2026-07-02T04:46:23.613Z)

```
Found 1 file
tests\test_storage_gdrive_api.py
```

8. user (2026-07-02T04:46:28.236Z)

```
1       """M12 ? the ``gdrive-api://`` OAuth/API StorageBackend (D14 Northstar).
2
3       Fully offline: a tiny in-memory ``_FakeDrive`` stands in for the googleapiclient Drive
v3 ``service``
4       (injected, mirroring the GCS fake-client tests). No ``google-api-python-client`` /
network needed.
5       Asserts: scheme parsing, registry wiring, the hyphen?underscore settings lookup, and the
full
6       path-keyed round-trip (put ? exists ? stat ? fetch ? list ? delete).
7       """
8
9       import re
10
11      import pytest
12
13      from backend.config import AppConfig
14      from backend.storage import get_storage
15      from backend.storage.gdrive_api import GDriveApiStorage
16      from backend.storage.ref import parse_uri
```

```
17
18      # ------------------------------------------------------------------ the in-memory fake
Drive service
19
20
21      class _Exec:
22          def __init__(self, value):
23              self._value = value
24
25          def execute(self):
26              return self._value
27
28
29      class _FakeFiles:
30          def __init__(self, drive):
31              self.d = drive
32
33          def list(self, q="", spaces=None, fields=None, pageToken=None):
34              name_m = re.search(r"name = '([^']*)'", q)
35              parent_m = re.search(r"'([^']*)' in parents", q)
36              parent = parent_m.group(1) if parent_m else None
37              out = []
38              for nid, n in self.d.nodes.items():
39                  if parent is not None and parent not in n["parents"]:
40                      continue
41                  if name_m and n["name"] != name_m.group(1):
42                      continue
43                  if nid == "root":
44                      continue
45                  out.append({"id": nid, "name": n["name"], "mimeType": n["mimeType"]})
46              return _Exec({"files": out})  # one page, no nextPageToken
47
48          def create(self, body=None, media_body=None, fields=None):
49              nid = self.d.new_id()
50              self.d.nodes[nid] = {
51                  "name": body["name"],
52                  "mimeType": body.get("mimeType", "application/octet-stream"),
53                  "parents": list(body.get("parents", [])),
54              }
55              if (
56                  media_body is not None
57              ):  # in tests _media_body is patched to return the local path
58                  with open(media_body, "rb") as fh:
59                      self.d.content[nid] = fh.read()
60              return _Exec({"id": nid})
61
62          def get_media(self, fileId=None):
63              return _Exec(self.d.content.get(fileId, b""))
64
65          def get(self, fileId=None, fields=None):
66              n = self.d.nodes[fileId]
67              data = self.d.content.get(fileId, b"")
68              return _Exec(
69                  {
70                      "size": str(len(data)),
71                      "modifiedTime": "2026-06-22T10:00:00.000Z",
72                      "md5Checksum": "deadbeef",
73                      "mimeType": n["mimeType"],
74                  }
75              )
76
77          def delete(self, fileId=None):
78              self.d.nodes.pop(fileId, None)
79              self.d.content.pop(fileId, None)
80              return _Exec({})
```

```
81
82
83      class _FakeDrive:
84          def __init__(self):
85              self.nodes = {"root": {"name": "root", "mimeType": "folder", "parents": []}}
86              self.content = {}
87              self._n = 0
88
89          def new_id(self):
90              self._n += 1
91              return f"id{self._n}"
92
93          def files(self):
94              return _FakeFiles(self)
95
96
97      @pytest.fixture
98      def be(monkeypatch):
99          """A GDriveApiStorage on a fake service, with _media_body patched to the local path
(no
100         googleapiclient dependency ? the fake `create` reads the bytes from that path)."""
101         monkeypatch.setattr(GDriveApiStorage, "_media_body", lambda self, p, ct: p)
102         return GDriveApiStorage(service=_FakeDrive())
103
104
105     # ------------------------------------------------------------------------ scheme +
registry wiring
106
107
108     def test_parse_uri_gdrive_api_is_path_keyed():
109         ref = parse_uri("gdrive-api://Coaching/2026/clip.mp4")
110         assert (ref.backend, ref.bucket, ref.key) == (
111             "gdrive-api",
112             "",
113             "Coaching/2026/clip.mp4",
114         )
115         assert ref.uri == "gdrive-api://Coaching/2026/clip.mp4"
116
117
118     def test_unknown_scheme_still_rejected():
119         with pytest.raises(ValueError):
120             parse_uri("gdrive-zzz://x")
121
122
123     def test_config_accepts_gdrive_api_backend():
124         cfg = AppConfig.model_validate(
125             {"storage": {"backend": "gdrive-api", "input_backend": "gdrive-api"}}
126         )
127         assert cfg.storage.backend == "gdrive-api"
128         assert cfg.storage.input_backend == "gdrive-api"
129
130
131     def test_get_storage_wires_gdrive_api_and_reads_underscore_settings(monkeypatch):
132         """Registry resolves 'gdrive-api' ? GDriveApiStorage, and its settings come from the
133         `gdrive_api` field (hyphen?underscore lookup) merged with overrides (here, the
injected service)."""
134         monkeypatch.setattr(GDriveApiStorage, "_media_body", lambda self, p, ct: p)
135         cfg = {"backend": "gdrive-api", "gdrive_api": {"root_folder_id": "FOLDER42"}}
136         be = get_storage("gdrive-api", config=cfg, service=_FakeDrive())
137         assert isinstance(be, GDriveApiStorage)
138         assert be._root_id == "FOLDER42"
139
140
141     # ------------------------------------------------------------------------------ full
round-trip
```

```
142
143
144     def test_roundtrip_put_exists_stat_fetch_list_delete(be, tmp_path):
145         src = tmp_path / "clip.mp4"
146         src.write_bytes(b"VIDEO-BYTES")
147         ref = parse_uri("gdrive-api://Coaching/2026/clip.mp4")
148
149         be.put(str(src), ref)  # creates the Coaching/ + 2026/ folders + the file
150         assert be.exists(ref)
151         assert be.stat(ref).size == len(b"VIDEO-BYTES")
152         assert be.stat(ref).etag == "deadbeef"
153
154         dst = tmp_path / "out" / "clip.mp4"
155         be.fetch_to_local(ref, str(dst))
156         assert dst.read_bytes() == b"VIDEO-BYTES"
157
158         listed = [r.uri for r in be.list(parse_uri("gdrive-api://Coaching/2026"))]
159         assert listed == ["gdrive-api://Coaching/2026/clip.mp4"]
160
161         be.delete(ref)
162         assert not be.exists(ref)
163
164
165     def test_put_is_idempotent_no_duplicate_on_reput(be, tmp_path):
166         """Re-putting the same path OVERWRITES (delete-then-create) ? Drive allows duplicate
names, so a
167         naive create would orphan the old file and reads would keep returning the stale
one."""
168         ref = parse_uri("gdrive-api://Coaching/clip.mp4")
169         v1 = tmp_path / "v1.mp4"
170         v1.write_bytes(b"FIRST")
171         v2 = tmp_path / "v2.mp4"
172         v2.write_bytes(b"SECOND-LONGER")
173         be.put(str(v1), ref)
174         be.put(str(v2), ref)  # re-put the SAME path
175         listed = [r.uri for r in be.list(parse_uri("gdrive-api://Coaching"))]
176         assert listed == ["gdrive-api://Coaching/clip.mp4"]  # exactly ONE file, not two
177         dst = tmp_path / "out.mp4"
178         be.fetch_to_local(ref, str(dst))
179         assert (
180             dst.read_bytes() == b"SECOND-LONGER"
181         )  # the latest bytes, not the orphaned first
182
183
184     def test_fetch_missing_raises(be, tmp_path):
185         with pytest.raises(FileNotFoundError):
186             be.fetch_to_local(
187                 parse_uri("gdrive-api://nope/missing.mp4"), str(tmp_path / "x")
188             )
189
190
191     def test_fetch_requires_dst(be):
192         with pytest.raises(ValueError):
193             be.fetch_to_local(parse_uri("gdrive-api://a/b.mp4"), None)
194
195
196     def test_put_at_root_uses_root_folder(be, tmp_path):
197         src = tmp_path / "top.json"
198         src.write_text("{}")
199         ref = parse_uri("gdrive-api://top.json")
200         be.put(str(src), ref)
201         assert be.exists(ref)
202
203
204     def test_build_service_without_libs_raises_actionable_error(monkeypatch):
```

```
205         """No injected service + the Google libs absent ? a clear, owner-actionable error
(not ImportError
206         leaking through). googleapiclient is not installed in the test env, so this
exercises the real path."""
207         with pytest.raises(RuntimeError, match="storage-gdrive-api"):
208             GDriveApiStorage()
209
```

9. user (2026-07-02T04:47:13.085Z)

```
backend\storage\s3.py:47:    def fetch_to_local(self, ref, dst=None, *, overwrite=False) -> str:
backend\storage\s3.py-48-        if dst is None:
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\storage\s3.py-49-
raise ValueError("S3Storage.fetch_to_local requires a dst path")
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\storage\s3.py-50-
dst = str(dst)
backend\storage\s3.py-51-        if os.path.exists(dst) and not overwrite:
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\storage\s3.py-52-
return dst
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\storage\s3.py-53-
os.makedirs(os.path.dirname(dst) or ".", exist_ok=True)
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\storage\s3.py-54-
self._client.download_file(ref.bucket, ref.key, dst)
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\storage\s3.py-55-
return dst
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\storage\s3.py-56-
backend\storage\s3.py-57-    def put(self, local_path, ref, *, content_type=None) -> StorageRef:
backend\storage\s3.py-58-        extra = {"ContentType": content_type} if content_type else None
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\storage\s3.py-59-
self._client.upload_file(str(local_path), ref.bucket, ref.key, ExtraArgs=extra)
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\storage\s3.py-60-
return ref
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\storage\s3.py-61-
backend\storage\s3.py-62-    def exists(self, ref) -> bool:
backend\storage\s3.py-63-        try:
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\storage\s3.py-64-
self._client.head_object(Bucket=ref.bucket, Key=ref.key)
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\storage\s3.py-65-
return True
backend\storage\s3.py-66-        except Exception:  # noqa: BLE001 - any boto/client error means
"not found / unreachable"
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\storage\s3.py-67-
return False
--
backend\storage\gcs.py:40:    def fetch_to_local(self, ref, dst=None, *, overwrite=False) ->
str:
backend\storage\gcs.py-41-        if dst is None:
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\storage\gcs.py-42-
raise ValueError("GCSStorage.fetch_to_local requires a dst path")
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\storage\gcs.py-43-
import os
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\storage\gcs.py-44-
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\storage\gcs.py-45-
dst = str(dst)
backend\storage\gcs.py-46-        if os.path.exists(dst) and not overwrite:
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\storage\gcs.py-47-
return dst
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\storage\gcs.py-48-
os.makedirs(os.path.dirname(dst) or ".", exist_ok=True)
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\storage\gcs.py-49-
self._blob(ref).download_to_filename(dst)
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\storage\gcs.py-50-
return dst
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\storage\gcs.py-51-
backend\storage\gcs.py-52-    def put(self, local_path, ref, *, content_type=None) ->
```

```
StorageRef:
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\storage\gcs.py-53-
self._blob(ref).upload_from_filename(str(local_path), content_type=content_type)
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\storage\gcs.py-54-
return ref
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\storage\gcs.py-55-
backend\storage\gcs.py-56-    def exists(self, ref) -> bool:
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\storage\gcs.py-57-
return bool(self._blob(ref).exists())
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\storage\gcs.py-58-
backend\storage\gcs.py-59-    def stat(self, ref) -> StorageStat:
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\storage\gcs.py-60-
b = self._blob(ref)
--
backend\storage\local.py:30:    def fetch_to_local(self, ref, dst=None, *, overwrite=False) ->
str:
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\storage\local.py-31-
# The object is already on the local FS ? return its path (identity). ``dst`` is
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\storage\local.py-32-
# honoured only if it names a *different* location (a copy is then made).
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\storage\local.py-33-
src = self._path(ref)
backend\storage\local.py-34-        if dst is None or os.path.abspath(str(dst)) ==
os.path.abspath(src):
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\storage\local.py-35-
return src
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\storage\local.py-36-
dst = str(dst)
backend\storage\local.py-37-        if os.path.exists(dst) and not overwrite:
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\storage\local.py-38-
return dst
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\storage\local.py-39-
os.makedirs(os.path.dirname(dst) or ".", exist_ok=True)
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\storage\local.py-40-
shutil.copyfile(src, dst)
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\storage\local.py-41-
return dst
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\storage\local.py-42-
backend\storage\local.py-43-    def put(self, local_path, ref, *, content_type=None) ->
StorageRef:
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\storage\local.py-44-
dst = self._path(ref)
backend\storage\local.py-45-        if os.path.abspath(str(local_path)) != os.path.abspath(dst):
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\storage\local.py-46-
os.makedirs(os.path.dirname(dst) or ".", exist_ok=True)
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\storage\local.py-47-
shutil.copyfile(str(local_path), dst)
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\storage\local.py-48-
return ref
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\storage\local.py-49-
backend\storage\local.py-50-    def exists(self, ref) -> bool:
--
backend\storage\base.py:23:    def fetch_to_local(
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\storage\base.py-24-
self,
backend\storage\base.py-25-        ref: StorageRef,
backend\storage\base.py-26-        dst: "str | os.PathLike[str] | None" = None,
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\storage\base.py-27-
*,
backend\storage\base.py-28-        overwrite: bool = False,
backend\storage\base.py-29-    ) -> str:
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\storage\base.py-30-
"""Make the object available as a local file; return its local path. For the
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\storage\base.py-31-
``local`` backend this is an identity passthrough (the object is already local)."""
```

```
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\storage\base.py-32-
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\storage\base.py-33-
@abstractmethod
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\storage\base.py-34-
def put(
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\storage\base.py-35-
self,
backend\storage\base.py-36-        local_path: "str | os.PathLike[str]",
backend\storage\base.py-37-        ref: StorageRef,
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\storage\base.py-38-
*,
backend\storage\base.py-39-        content_type: "str | None" = None,
backend\storage\base.py-40-    ) -> StorageRef:
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\storage\base.py-41-
"""Upload/copy a local file to ``ref``; return ``ref``."""
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\storage\base.py-42-
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\storage\base.py-43-
@abstractmethod
--
backend\storage\gdrive_api.py:132:    def fetch_to_local(self, ref, dst=None, *,
overwrite=False) -> str:
backend\storage\gdrive_api.py-133-        if dst is None:
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\storage\gdrive_api.py-134-            raise ValueError(
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\storage\gdrive_api.py-135-                "GDriveApiStorage.fetch_to_local
requires a dst path (no local mount)"
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\storage\gdrive_api.py-136-            )
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\storage\gdrive_api.py-137-        dst = str(dst)
backend\storage\gdrive_api.py-138-        if os.path.exists(dst) and not overwrite:
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\storage\gdrive_api.py-139-            return dst
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\storage\gdrive_api.py-140-        file_id = self._resolve_id(ref.key)
backend\storage\gdrive_api.py-141-        if not file_id:
backend\storage\gdrive_api.py-142-            raise FileNotFoundError(f"gdrive-api: {ref.uri!r}
not found")
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\storage\gdrive_api.py-143-        data = (
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\storage\gdrive_api.py-144-
self._service.files().get_media(fileId=file_id).execute()
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\storage\gdrive_api.py-145-        )  # alt=media ? raw bytes
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\storage\gdrive_api.py-146-        os.makedirs(os.path.dirname(dst) or ".",
exist_ok=True)
backend\storage\gdrive_api.py-147-        with io.open(dst, "wb") as fh:
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\storage\gdrive_api.py-148-            fh.write(data)
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\storage\gdrive_api.py-149-        return dst
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\storage\gdrive_api.py-150-
backend\storage\gdrive_api.py-151-    def put(self, local_path, ref, *, content_type=None) ->
StorageRef:
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\storage\gdrive_api.py-152-        parent_path, name = self._split(ref.key)
--
backend\storage\gdrive.py:123:    def fetch_to_local(self, ref, dst=None, *, overwrite=False) ->
str:
backend\storage\gdrive.py-124-        # The file is already local (via the mount): no dst =>
identity passthrough to the mount
```

```
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\storage\gdrive.py-125-        # path; a dst => copy out (a byte-stable scratch
copy), exactly as the local backend does.
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\storage\gdrive.py-126-        self._require_present(ref)
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\storage\gdrive.py-127-        return self._local.fetch_to_local(
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\storage\gdrive.py-128-            self._local_ref(ref), dst, overwrite=overwrite
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\storage\gdrive.py-129-        )
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\storage\gdrive.py-130-
backend\storage\gdrive.py-131-    def put(self, local_path, ref, *, content_type=None) ->
StorageRef:
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\storage\gdrive.py-132-        self._local.put(local_path, self._local_ref(ref),
content_type=content_type)
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\storage\gdrive.py-133-        return ref
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\storage\gdrive.py-134-
backend\storage\gdrive.py-135-    def exists(self, ref) -> bool:
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\storage\gdrive.py-136-        return self._local.exists(self._local_ref(ref))
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\storage\gdrive.py-137-
backend\storage\gdrive.py-138-    def stat(self, ref) -> StorageStat:
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\storage\gdrive.py-139-        self._require_present(ref)
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\storage\gdrive.py-140-        return self._local.stat(self._local_ref(ref))
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\storage\gdrive.py-141-
backend\storage\gdrive.py-142-    def list(self, prefix) -> Iterator[StorageRef]:
backend\storage\gdrive.py-143-        for r in self._local.list(self._local_ref(prefix)):
```

10. user (2026-07-02T04:47:14.265Z)

```
b6214e5 chore: ruff format the whole repo (259 files) (#387)
f523d3a refactor: enable isort (I) in ruff, auto-fix 143 files (Q6) (#360)
819e395 feat(serving): M12 ? gdrive-api:// OAuth/API StorageBackend (D14 Northstar) (#338)
---
e410136 api: /api/videos surfaces fps + resolution (from the stored ffprobe details) (#413)
81a78bf api: /api/videos returns video_id + a friendly match_name (fix disappearing name) (#412)
8b16cf4 docs: reflect today's storage endpoints + the corpus/user-video boundary (#411)
aa915a7 deploy/gcp: centralize project + bucket names in one sourced buckets.env (#410)
52d97b5 P4: POST /api/assets ? store a video into the user's chosen medium (#409)
7ea6ea9 P3: /api/capabilities reports usable storage backends (#408)
f28af73 P2: engine free-space guard (HTTP 507 + WSL frame-stage BLOCK) (#407)
759a67f feat(api): frame-preserving annotation proxy for /api/source (D4) (#406)
5a5c4fa feat(api): P8a ? GET /api/source + POST /api/annotations (annotate-in-UI backend) (#405)
4a7bfbe docs: record the launch NO-GO decision (owner, 2026-06-30) (#404)
```

11. user (2026-07-02T04:47:21.522Z)

```
mypy.ini
pyproject.toml
ruff.toml
origin  https://github.com/Khelsutra/badminton-highlight-indexer.git (fetch)
origin  https://github.com/Khelsutra/badminton-highlight-indexer.git (push)
```

12. user (2026-07-02T04:47:22.683Z)

20:## 0. REPO LAYOUT ? where new files go (READ BEFORE committing a new file)

13. user (2026-07-02T04:47:28.107Z)

```
20      ## 0. REPO LAYOUT ? where new files go (READ BEFORE committing a new file)
21
22      Top-level tree and the **placement rule** for each kind of file. When unsure, prefer an
existing
23      **registry / extension point** (`?`) over a new top-level file.
24
25      | You're adding? | Put it in? | Notes |
26      |---|---|---|
27      | Backend / runtime code | `backend/<subsystem>/` |
`pipeline/{segmenters,detectors,validators}`, `providers/`, `sports/`, `annotations/`,
`infrastructure/`, `reporting/`, `stitcher/`, `utils/`, `config/`, `features/`, `api/`. Most
additions are **register a class in the relevant `?` registry**, not a new module tree. |
28      | **Eval LIBRARY** (importable, pure ? metrics, calibration, a scorer, a feature) |
`backend/eval/` | Pure core, no side-effects at import; CLI/I-O lives in a separate driver.
These are **imported widely** (e.g. `calibrate_wasb` by 8 sites) ? they are library, not
scripts; do NOT move them out. |
29      | **Standalone run-driver / one-off entry CLI** (has `__main__`, NOT imported by app
code) | **`scripts/`** | e.g. `scripts/run_phase3_eval.py`, `scripts/run_process_and_stitch.py`.
Run as `python scripts/<name>.py`. If it needs `load_dotenv()`/`sys.path` before importing
backend, add `"scripts/<name>.py" = ["E402"]` to `ruff.toml`. |
30      | **Ops / maintenance tool** (not part of the served app ? cache GC, batch admin) |
`tools/` | Run as `python -m tools.<name>` (it's an importable package). e.g.
`tools/cleanup_caches.py`. Keep the destructive decision path pure + unit-tested. |
31      | Training code (model fitting) | `training/` | Behind the CI-enforced import wall
(ADR-008) ? `backend.main` must not import it. Own `requirements-train.txt`. |
32      | A test | `tests/test_*.py` | pytest auto-discovers; the full-suite lane has a **70%
coverage floor**. Pure/offline + mocked (no GPU/WSL/network). |
33      | A doc (design, plan, living log) | `docs/` | Index it in `docs/README.md`. Only
**terminal-state** (DONE/REJECTED) work moves to `docs/archives/`. |
34      | A throwaway repro / debug script | `scratch/` | **Gitignored** + excluded from
ruff/pytest. Never imported by app code. |
35      | A model artifact (weights + manifest) | `artifacts/<name>/<ver>/` | ADR-008:
**manifest committed** (provenance), **weights gitignored**. |
36      | Eval baseline / leaderboard JSON | `eval_baselines/` | Tracked (survives a clean
checkout); the no-regression gate reads it. |
37      | **Committed regression fixture** (video-free golden) |
`eval_baselines/fixtures/<slug>/` | Tracked, **LF-pinned** (`.gitattributes`). Synthetic Tier-0
(or owner-cleared, de-attributed real). Drives the `golden_fixtures` characterization gate with
**no video/GPU/DB**. See `docs/GOLDEN_REGRESSION_FIXTURES.md`; mint via `python -m
backend.eval.golden_fixtures --freeze-synthetic` / `--freeze-real` (refusal-gated). |
38      | Pipeline run outputs (reels, trajectories, DBs, frames) | `output/` | **Gitignored.**
Per-video isolation is the tracked **`PER_VIDEO_WORKSPACE.md`** project (P0 helper shipped #264;
supersedes the archived `PER_VIDEO_OUTPUT_LAYOUT`). |
39
40      **Canonical files that stay at the repo ROOT** (referenced by CI / packaging / docs ? do
not move):
41      `pyproject.toml` (PEP 621 packaging, hatchling; `indexer` entry point) ·
`run_all_tests.py` (CI: `ci.yml`) ·
42      `run_regression.py` (the no-regression gate CLI, cited in `EXPERIMENT_HARNESS.md`) ·
`run_eval.py`
43      (grandfathered ? imported by `tests/test_eval_harness.py`). The old diagnostic
`setup.py` moved to
44      `scripts/doctor.py` (it shadowed the build system); invoke via `python
scripts/doctor.py` or `indexer --setup`.
45      New standalone run-drivers go in **`scripts/`**, not the root.
46
47      ---
48
49      ## 1. PIPELINE
50
51      ### 1A. Runtime: video file -> highlight reel
```

```
52      ```
53      typed config load (built-in defaults -> config.json overrides; absent/parse-fail -> all-
defaults, never fatal)
54                                                  @backend/config/loader.py::load_config ->
@backend/config/models.py::AppConfig
55        -> video_id = MD5(whole file)           @backend/utils/hashing.py::compute_video_id
(single shared helper, 64KB chunks; imported by main + all run_*.py ? no more per-entrypoint
copies)
56        -> validate
@backend/pipeline/validators/base.py::registry.run_all_with_context(video_path, global_config)
57              FormatValidator -> ctx['ffprobe_meta'] -> ResolutionFPSValidator -> PTSContinuity
-> SceneCut -> Blur -> Relevance   (? ACTUAL registration/exec order ? see
@backend/pipeline/validators/__init__.py)
58              [validation FAIL -> persist status="failed", return early; report STILL emitted
in finally]
59        -> db.add_video(video_id, filepath, status, [r.dict()])
@backend/infrastructure/database.py
```

14. assistant (2026-07-02T04:47:33.751Z)

Placement rules are clear ? I'm only editing existing files. Creating the isolated worktree now.

15. user (2026-07-02T04:47:38.538Z)

```
Preparing worktree (new branch 'fix/gdrive-api-chunked-download-safe-put')
branch 'fix/gdrive-api-chunked-download-safe-put' set up to track 'origin/master'.
HEAD is now at e410136 api: /api/videos surfaces fps + resolution (from the stored ffprobe
details) (#413)
fix/gdrive-api-chunked-download-safe-put
e410136 api: /api/videos surfaces fps + resolution (from the stored ffprobe details) (#413)
```

16. assistant (2026-07-02T04:48:37.721Z)

Worktree ready on `fix/gdrive-api-chunked-download-safe-put` at origin/master. Now implementing the fix in the worktree copy of `gdrive_api.py`.

17. user (2026-07-02T04:48:39.937Z)

```
1       """Google Drive **API** backend (OAuth / service-account) ? ``gdrive-api://`` (M12, D14
Northstar).
2
3       The cloud-native Drive round-trip: read a source from + write the stitched reel **back
to** a user's
4       Google Drive **via the Drive API** ? no local mount required (unlike the ``gdrive://``
backend, which
5       is plain local-FS over a *mounted* Drive for Desktop and stays untouched). This is the
headless path a
6       Cloud Run / GPU box uses.
7
8       ``gdrive-api://<path>`` names a path **relative to a root folder** (default the Drive
``root``, or
9       ``storage.gdrive-api.root_folder_id``); the backend resolves each path component to a
Drive file id via
10      ``files().list``. The Google libraries (``google-api-python-client`` + ``google-auth``)
are imported
11      **lazily** ? ``import backend.storage`` never pulls them ? and a ``service`` may be
**injected** for
12      fully-offline tests (mirroring ``GCSStorage(client=...)``).
13
14      ? **Credentials are never in config.** The real service is built from Application
Default Credentials
15      (a service account / workload identity) over the ``drive`` scope. **Per-user interactive
OAuth (the
16      consumer "connect your Drive" flow: client id/secret + refresh tokens) is OWNER-gated**
```

```
? this backend
17      ships the service-account/ADC build + the injected fake; the live per-user OAuth app is
a follow-up.
18
19      **Scope caveats (M12):** handles **binary** files (videos / reels / JSON) ? Google-
native Docs
20      (Sheets/Slides) have no direct download and would need ``files().export_media`` (out of
scope; export
21      them first). ``put`` is idempotent (deletes an existing file at the same path before
creating). Path
22      resolution keys on ``(name, parent)``; if Drive already holds duplicate-named files
under one folder
23      (a rare pre-existing state this backend never creates), resolution returns the first
match.
24      """
25
26      from __future__ import annotations
27
28      import io
29      import os
30      from datetime import datetime
31      from typing import Any, Iterator, Optional
32
33      from backend.storage.base import StorageBackend
34      from backend.storage.ref import StorageRef, StorageStat
35
36      _FOLDER_MIME = "application/vnd.google-apps.folder"
37      _DRIVE_SCOPE = "https://www.googleapis.com/auth/drive"
38
39
40      def _q_escape(name: str) -> str:
```

18. user (2026-07-02T04:48:51.003Z)

```
The file C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-
indexer/c1fcb4aa-537d-41fd-858e-53f93349bf5a/scratchpad/wt-gdrive-api-
safety/backend/storage/gdrive_api.py has been updated successfully. (file state is current in
your context ? no need to Read it back)
```

19. user (2026-07-02T04:48:59.332Z)

```
The file C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-
indexer/c1fcb4aa-537d-41fd-858e-53f93349bf5a/scratchpad/wt-gdrive-api-
safety/backend/storage/gdrive_api.py has been updated successfully. (file state is current in
your context ? no need to Read it back)
```

20. assistant (2026-07-02T04:49:03.585Z)

Now the `_downloader` seam (mirroring `_media_body`) and the new `fetch_to_local`.

21. user (2026-07-02T04:49:07.881Z)

```
The file C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-
indexer/c1fcb4aa-537d-41fd-858e-53f93349bf5a/scratchpad/wt-gdrive-api-
safety/backend/storage/gdrive_api.py has been updated successfully. (file state is current in
your context ? no need to Read it back)
```

22. user (2026-07-02T04:49:18.675Z)

```
The file C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-
indexer/c1fcb4aa-537d-41fd-858e-53f93349bf5a/scratchpad/wt-gdrive-api-
safety/backend/storage/gdrive_api.py has been updated successfully. (file state is current in
your context ? no need to Read it back)
```

23. assistant (2026-07-02T04:49:23.979Z)

Now the `put()` reorder ? update-in-place per the verifier's refinement.

24. user (2026-07-02T04:49:30.835Z)

The file C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-indexer/c1fcb4aa-537d-41fd-858e-53f93349bf5a/scratchpad/wt-gdrive-api-safety/backend/storage/gdrive_api.py has been updated successfully. (file state is current in your context ? no need to Read it back)

25. assistant (2026-07-02T04:49:38.562Z)

Now extend the tests in the worktree.

26. user (2026-07-02T04:49:43.576Z)

<tool_use_error>File has not been read yet. Read it first before writing to it.</tool_use_error>

27. user (2026-07-02T04:49:55.576Z)

```
1       """M12 ? the ``gdrive-api://`` OAuth/API StorageBackend (D14 Northstar).
2
3       Fully offline: a tiny in-memory ``_FakeDrive`` stands in for the googleapiclient Drive
v3 ``service``
4       (injected, mirroring the GCS fake-client tests). No ``google-api-python-client`` /
network needed.
5       Asserts: scheme parsing, registry wiring, the hyphen?underscore settings lookup, and the
full
6       path-keyed round-trip (put ? exists ? stat ? fetch ? list ? delete).
7       """
8
9       import re
10
11      import pytest
12
13      from backend.config import AppConfig
14      from backend.storage import get_storage
15      from backend.storage.gdrive_api import GDriveApiStorage
```

28. user (2026-07-02T04:50:03.789Z)

The file C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-indexer/c1fcb4aa-537d-41fd-858e-53f93349bf5a/scratchpad/wt-gdrive-api-safety/tests/test_storage_gdrive_api.py has been updated successfully. (file state is current in your context ? no need to Read it back)

29. user (2026-07-02T04:50:11.566Z)

The file C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-indexer/c1fcb4aa-537d-41fd-858e-53f93349bf5a/scratchpad/wt-gdrive-api-safety/tests/test_storage_gdrive_api.py has been updated successfully. (file state is current in your context ? no need to Read it back)

30. user (2026-07-02T04:50:22.493Z)

The file C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-indexer/c1fcb4aa-537d-41fd-858e-53f93349bf5a/scratchpad/wt-gdrive-api-safety/tests/test_storage_gdrive_api.py has been updated successfully. (file state is current in your context ? no need to Read it back)

31. user (2026-07-02T04:50:41.028Z)

```
The file C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-
indexer/c1fcb4aa-537d-41fd-858e-53f93349bf5a/scratchpad/wt-gdrive-api-
safety/tests/test_storage_gdrive_api.py has been updated successfully. (file state is current in
your context ? no need to Read it back)
```

32. user (2026-07-02T04:51:02.315Z)

```
The file C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-
indexer/c1fcb4aa-537d-41fd-858e-53f93349bf5a/scratchpad/wt-gdrive-api-
safety/tests/test_storage_gdrive_api.py has been updated successfully. (file state is current in
your context ? no need to Read it back)
```

33. assistant (2026-07-02T04:51:06.867Z)

Now the download-path tests, after `test_fetch_missing_raises` area.

34. user (2026-07-02T04:51:17.357Z)

```
The file C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-
indexer/c1fcb4aa-537d-41fd-858e-53f93349bf5a/scratchpad/wt-gdrive-api-
safety/tests/test_storage_gdrive_api.py has been updated successfully. (file state is current in
your context ? no need to Read it back)
```

35. user (2026-07-02T04:51:28.418Z)

```
..............                                                           [100%]
14 passed in 0.54s
```

36. user (2026-07-02T04:51:35.834Z)

```
Exit code 1
All checks passed!
Would reformat: tests\test_storage_gdrive_api.py
1 file would be reformatted, 1 file already formatted
```

37. user (2026-07-02T04:51:46.756Z)

```
1 file reformatted
 backend/storage/gdrive_api.py    |  68 ++++++++++++-----
 tests/test_storage_gdrive_api.py | 152 ++++++++++++++++++++++++++++++++++++++--
 2 files changed, 199 insertions(+), 21 deletions(-)
Success: no issues found in 1 source file
[This command modified 1 file you've previously read: ..\..\..\AppData\Local\Temp\claude\C--
Users-avidu-Projects-badminton-highlight-
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-gdrive-api-
safety\tests\test_storage_gdrive_api.py. Call Read before editing.]
```

38. assistant (2026-07-02T04:51:52.052Z)

Ruff reformatted one comment wrap. Re-running tests to confirm, then committing.

39. user (2026-07-02T04:51:57.725Z)

```
..............                                                           [100%]
14 passed in 0.38s
All checks passed!
2 files already formatted
fix/gdrive-api-chunked-download-safe-put
e410136 api: /api/videos surfaces fps + resolution (from the stored ffprobe details) (#413)
 M backend/storage/gdrive_api.py
 M tests/test_storage_gdrive_api.py
```

40. user (2026-07-02T04:52:07.944Z)

```
Found 1 file
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-gdrive-api-
safety\tests\test_storage_gdrive_api.py
```

41. user (2026-07-02T04:52:08.454Z)

```
Found 3 files
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-gdrive-api-
safety\backend\storage\gdrive_api.py
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-gdrive-api-
safety\backend\storage\__init__.py
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-gdrive-api-
safety\backend\config\models.py
```

42. user (2026-07-02T04:52:29.019Z)

```
.....................................................                [100%]
57 passed, 2 skipped, 1505 deselected in 11.95s
```

43. assistant (2026-07-02T04:52:33.191Z)

```
All storage tests pass. Committing with explicit paths.
```

44. user (2026-07-02T04:52:44.220Z)

```
fix/gdrive-api-chunked-download-safe-put
e410136 api: /api/videos surfaces fps + resolution (from the stored ffprobe details) (#413)
[fix/gdrive-api-chunked-download-safe-put 6afacdd] fix(storage): gdrive-api ? chunked streaming
download + loss-proof idempotent put
 2 files changed, 199 insertions(+), 21 deletions(-)
6afacdd fix(storage): gdrive-api ? chunked streaming download + loss-proof idempotent put
```

45. user (2026-07-02T04:52:55.538Z)

```
branch 'fix/gdrive-api-chunked-download-safe-put' set up to track 'origin/fix/gdrive-api-
chunked-download-safe-put'.
To https://github.com/Khelsutra/badminton-highlight-indexer.git
 * [new branch]      fix/gdrive-api-chunked-download-safe-put -> fix/gdrive-api-chunked-
download-safe-put
```

46. user (2026-07-02T04:53:19.205Z)

```
https://github.com/Khelsutra/badminton-highlight-indexer/pull/416
```

47. assistant (2026-07-02T04:53:33.896Z)

```
PR opened. Returning the structured result.
```

48. user (2026-07-02T04:53:47.741Z)

```
Structured output provided successfully
```